

2024 DeepFlow 在 TKE 内部平台的 可观测性实践

赵奇圆 腾讯云高级工程师



中国计算机学会
CHINA COMPUTER FEDERATION

DeepFlow



蓝鲸智云



ClickHouse

eBPF 零侵入可观测性 Meetup

CONTENTS

- 01 TKE 内部平台可观测性挑战
- 02 为什么选用 DeepFlow
- 03 落地与优化实践
- 04 案例分享
- 05 未来展望

01、TKE 内部平台可观测性挑战

随着云原生技术发展，越来越多业务拥抱微服务化架构和 K8S 平台，这显著提升了业务研发效率，但在可观测性方面带来了诸多挑战

01 TKE 内部平台可观测挑战

TKE 内部平台介绍:

以腾讯云容器服务 (Tencent Kubernetes Engine, TKE) 为底座, 服务于腾讯自研业务的容器平台

挑战一: 微服务化引入的挑战

- 服务模块变多, 依赖关系复杂
- 生命周期变短, 网络拓扑结构动态变化, 传统的静态拓扑图不能体现业务真实网络拓扑

挑战二: K8S化引入的挑战

- 网络链路变长, 自动化运维工具没跟上

挑战三: 推动业务改造成本高

- 业务技术栈差异较大 (C++/Go/Python/Java等)
- 难以完全覆盖所有网络实体

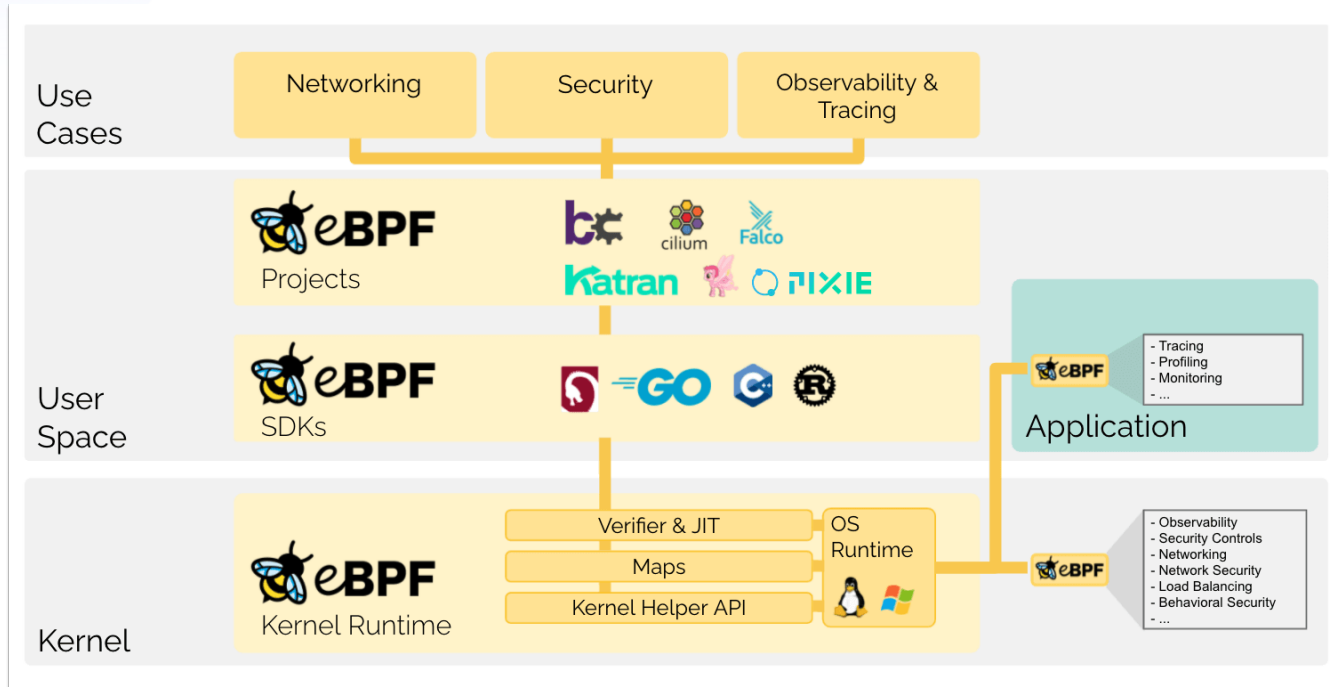
功能: 包含数据采集, 存储 (可回溯) 和展示 (链路拓扑) 的可观测系统; 不依赖业务改造 (能落地), 从而实现高效率的故障排查

可用性: 对业务性能的影响要足够小, 资源占用足够少, 能实现长稳运行

02、为什么选用 DeepFlow

怎么实现零侵入和高性能的数据采集能力，并且具备一整套可观测解决方案

零侵入，高性能的数据采集能力



原理：内核层面的扩展能力，内核实现了支持 eBPF 程序运行的框架（加载点，校验机制，加载和运行流程等）

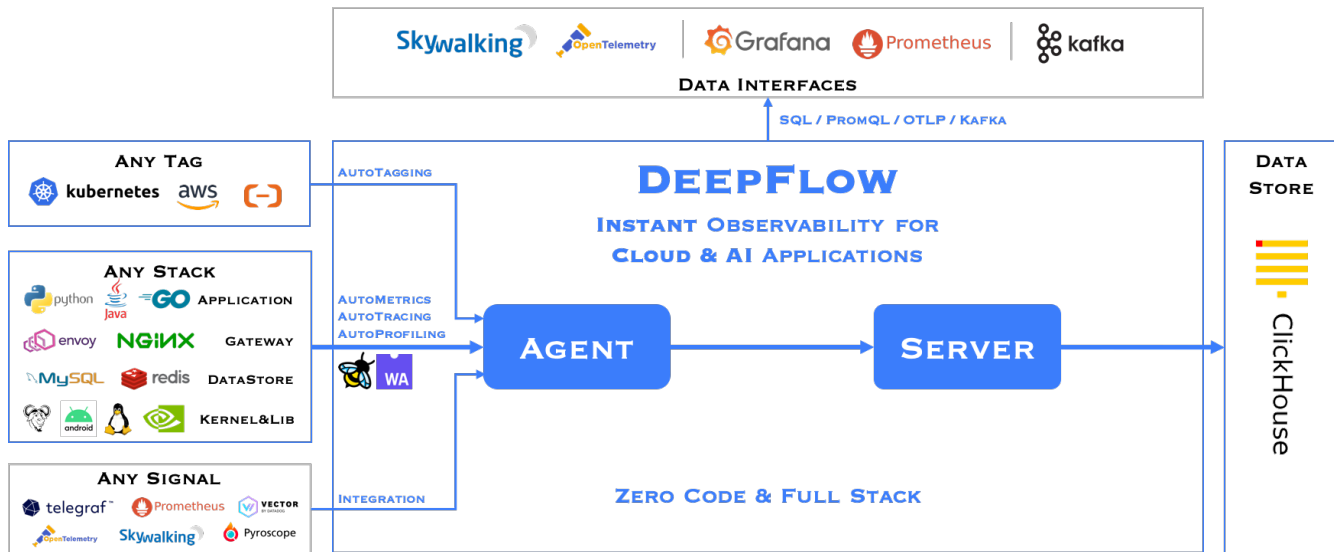
能力：业务方不修改内核源码即可以实现一些网络和安全功能，增强系统可观测性

类似于 K8S 中 CNI，CSI 等可扩展机制

02 采集，存储，展示和分析的可观测系统

业界 eBPF 的可观测性方案中，Cilium/Hubble、Pixie 和 DeepFlow 相对成熟。但 Hubble 强依赖于 Cilium，落地成本较高；Pixie 缺乏完整的数据采集、存储和展示体系

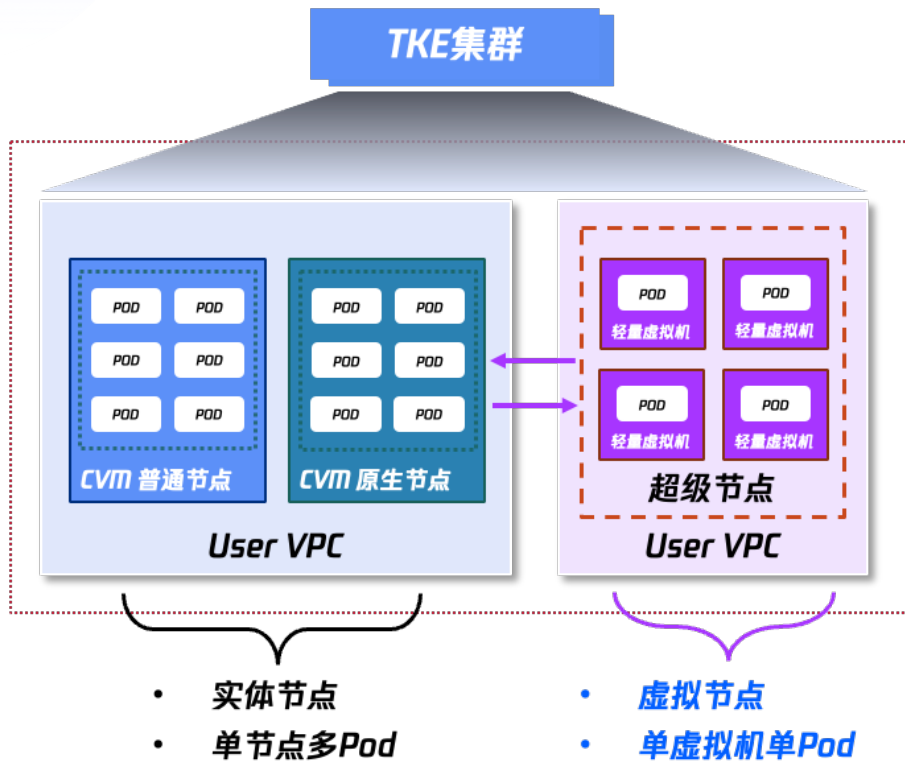
DeepFlow 提供了全面的解决方案（涵盖网络层、应用层指标与流数据、服务拓扑以及调用链跟踪），其采集端采用 Rust 编写，性能表现良好，同时社区活跃，并有大规模生产环境的成功实践



03、落地与优化实践

业务大量跑在 TKE Serverless 容器服务上，并且采用 StatefulSetPlus 管理，DeepFlow 需要做哪些适配

怎么支持 TKE Serverless 容器服务



TKE Serverless 容器服务

产品形态：超级节点，没有对应的实体节点，可以视为一个虚拟资源池

底层实现：一个 Pod 对应一个轻量虚拟机

优势：隔离性高，免除节点运维负担

业务情况：腾讯会议等自研业务

DeepFlow 接入方式

普通节点/原生节点：默认模式启动；Daemonset 部署

超级节点：Sidecar 模式启动；采用超级节点 Daemonset 注入方式部署，**每一个 Pod 都要启用一个 Agent**

Agent 内存占用

挑战一：不可压缩资源，影响业务稳定性

挑战二：大规模部署成本

目标：Agent 内存占用控制在100M

Agent 内存优化实践

初始阶段

数据：内存占用约为260M

分析：cBPF 模块为主要内存消耗来源，而 eBPF 模块占用较小

优化方向：初期只开启 eBPF 功能，支持应用层指标，流日志（HTTP/DNS/GRPC等）及链路拓扑观察的需求

阶段一：优化 Agent 内存

问题：关闭 cBPF，Agent 内存并没有下降

[解决方案](#)：反馈社区，cBPF 不开启时 cBPF 模块不启用

数据：260M -> 60M

阶段二：优化 eBPF map内存

问题：部署前后多消耗了约140M，其中80M来自 eBPF Map

[解决方案](#)：关闭部分无关功能（on-cpu-profile，off-cpu-profile）；针对 Serverless 容器单Pod的特点，缩小 map 的配置项（如 max-trace-entries）

数据：80M -> 35M

内存占用最终从340M -> 95M

03 Agent 配置说明

```
1 max_millicpus: 500
2 max_memory: 256
3 system_load_circuit_breaker_threshold: 2
4 system_load_circuit_breaker_recover: 1.8
5 sys_free_memory_limit: 10
6 sys_free_memory_metric: available
7 tap_interface_regex: ^notexit
8 vtap_flow_1s_enabled: 1
9 l7_log_collect_nps_threshold: 20000
10 external_agent_http_proxy_enabled: 0
11 static_config:
12   ebp:
13     perf-pages-count: 32
14     ring-size: 8192
15     max-trace-entries: 30000
16     io-event-collect-mode: 0
17     on-cpu-profile:
18       disabled: true
19     off-cpu-profile:
20       disabled: true
21     ebp-collector-queue-size: 4096
```

熔断

内存优化

内存熔断机制

目的：防止内存紧张时 Agent 与业务容器争抢资源

说明：系统可用内存低于10%，进入静默模式

优化：最初判断基于 free 内存，部分业务因 buf/cache 占用过多而产生误判。推动社区支持了基于 available 内存判断，从而减少误告

降低了 Agent 在 Serverless 容器场景下的内存占用和资源竞争风险，提升了系统的整体稳定性与成本效益

怎么支持自定义 Workload

StatefulSetPlus

介绍：TKE自研，用于管理有状态服务的 Operator

核心能力：

- 支持自动以及手动的分批灰度发布能力
- 支持 Pod 原地升级
- 对接了 TKE 网络插件，支持 Pod 固定 IP
- 单个 StatefulSetPlus 实例可管理上万个 Pod

问题：

DeepFlow 默认并不识别 StatefulSetPlus。当 DeepFlow 运行在 Serverless 容器上时，由于无法识别这些 Pod 所属的控制器类型，Agent 无法完成注册

解决方案：

- 定义 StatefulSetPlus 对应的 CRD 信息
- 注册并监听该 CRD 资源

```

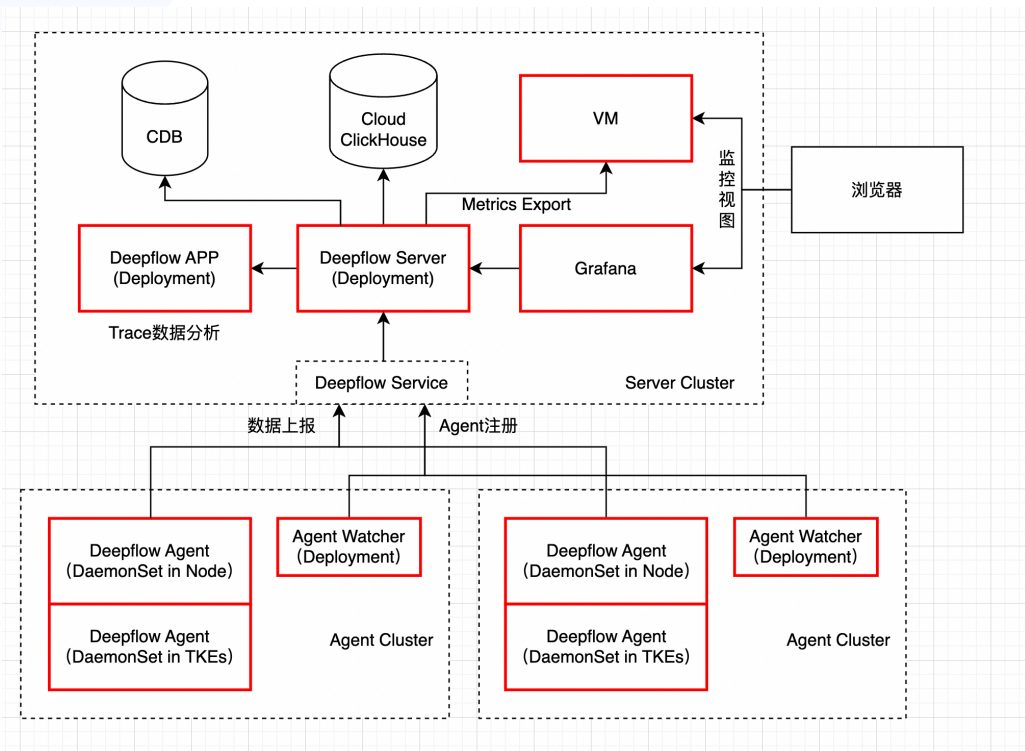
agent/src/platform/kubernetes/resource_watcher.rs
15 use super::crd::({
59     krusei::({CloneSet, StatefulSet as KruseiStatefulSet},
60     opengauss::OpenGaussCluster,
61     pingan_cloud::ServiceRule,
62     tke::StatefulSetPlus,
63 });
64 use crate::utils::stats::({self, Countable, Counter, CounterType, CounterValue, RefCountable});
65

querier/engine/clickhouse
clickhouse_test.go
filter.go
group.go
tag
translation.go

181 182 KruseiStatefulSet(ResourceMatcher=>KruseiStatefulSet),
182 183 {ipPool(ResourceMatcher=>IpPool)},
183 184 OpenGaussCluster(ResourceMatcher=>OpenGaussCluster)},
185 + StatefulSetPlus(ResourceMatcher=>StatefulSetPlus)},
184 186 }
185 187 #[derive(Clone, Copy, Debug, PartialEq, Eq)]
186 188
344 346 group: "apps.krusei.io",
345 347 version: "v1beta1",
346 348 },
349 + GroupVersion {
350 + group: "platform.stke",
351 + version: "v1alpha1",
352 + },
353 + },
354 selected_gv: None,
355 field_selector: String::new(),
356

1380 1386 namespace,
1381 1387 config,
1382 1388 })),
1389 + GroupVersion {
1390 + group: "platform.stke",
1391 + version: "v1alpha1",
1392 + } => GenericResourceMatcher::StatefulSetPlus(self.new_watcher_inner(
1393 + resource,
1394 + stats_collector,
1395 + namespace,
1396 + config,
1397 + )),
1383 1398 GroupVersion {
1384 1399 group: "apps",
1385 1400 version: "v1",
1401

37 37 "InPlaceSet": false,
38 38 "ReplicaSet": false,
39 39 "StatefulSet": false,
40 + "StatefulSetPlus": false,
40 41 "OpenGaussCluster": false,
41 42 "ReplicationController": false,
42 43 }
  
```

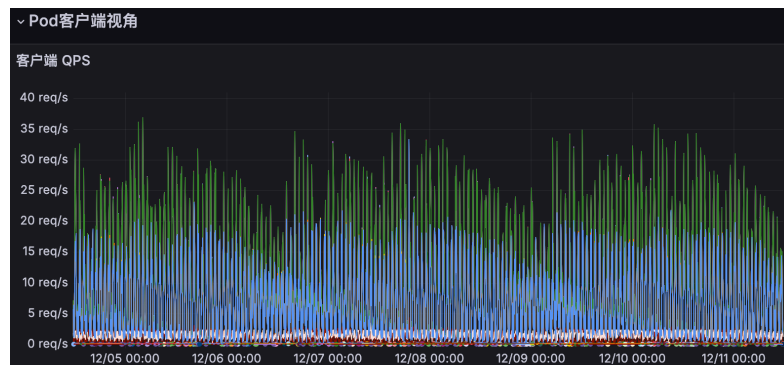
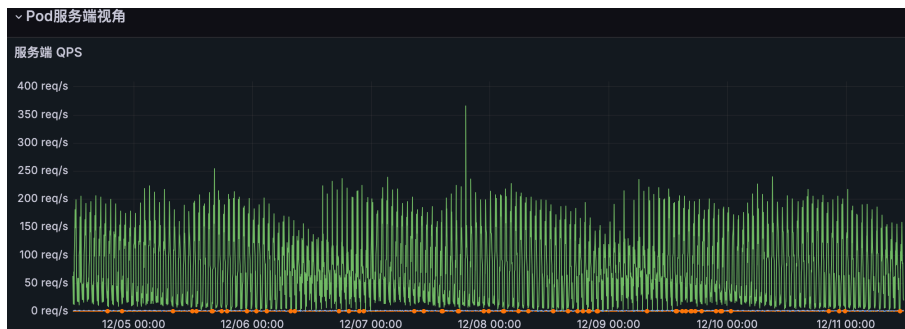
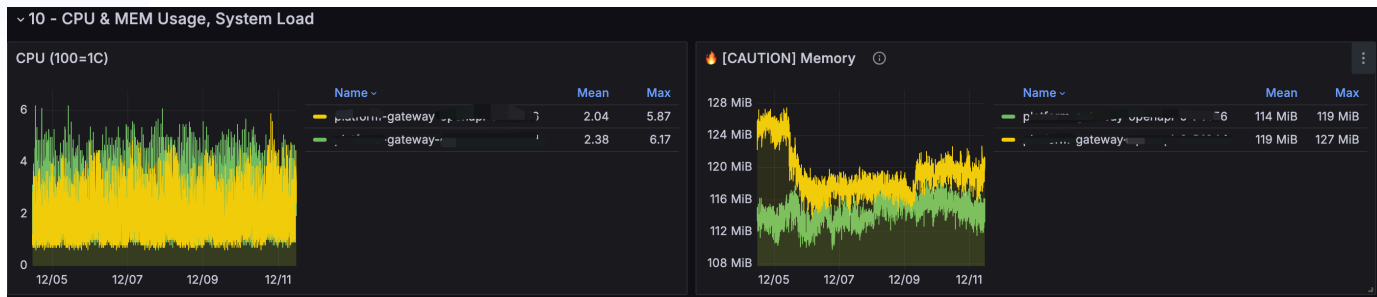


Server

- 一套 Server 纳管多套集群
- 引入了 VM, CK 数据保留3天, 指标数据通过 Remote Write 能力同步到 VM, 保留14天

Agent

- 普通节点和超级节点 (关闭 cBPF) Agent 配置差异较大, 采用不同配置和 Daemonset
- 默认部署模式下, K8S APIWatcher 是从 Agent 中选举出来, 这导致 Agent 和 APIWatcher 的可用性都无法保证, 推动社区针对 Agent 支持禁用 K8S APIWatcher, 从 Agent 中独立出来



内存占用：单 Pod 入向 QPS (HTTP 短连接) 最大 200，出向 30，Agent 7 天内 CPU 占用最大 0.06C，内存占用最大 127M

04、案例分享

排障加速



中国计算机学会
CHINA COMPUTER FEDERATION

DeepFlow



蓝鲸智云



ClickHouse

eBPF 零侵扰可观测性 Meetup

异常	计划/执行	状态
id":"1616063",...	Post "http://cloud-app.my...	2024-05-20 11:45:44 运行失败 C R
cess_inst_id":"...	Post "http://cloud-app.my... : 15000/api/callback/workitem/create": dial tcp: lookup cloud-app. on 10.10.10.255.254:53: read udp 10.10.10.25.242:49765->10.10.10.255.254:53: i/o timeout	
anageSrv","pro...		
_id":"2024053...	Post "http://cloud-app.my...	2024-05-20 11:45:44 运行失败 C R
change_systo...	Post "http://cloud-app.my...	2024-05-20 11:45:44 运行失败 C R
w","process_i...	Post "http://cloud-app.my...	2024-05-20 11:45:44 运行失败 C R
w","process_i...	Post "http://cloud-app.my...	2024-04-29 20:12:46 运行失败 C R
rocess_inst_i...	Post "http://cloud-app.my...	2024-04-29 20:12:46 运行失败 C R

问题

业务反馈服务高峰期会出现 DNS 解析偶发失败，持续时间1个小时，业务要求明确具体原因和解决方案，业务和 CoreDNS 部署在普通节点上

链路

客户端Pod->客户端节点->母机->CoreDNS Pod节点->CoreDNS Pod

排障加速

1. 先让客户提供客户端 Pod，因为缺失现场，不能明确是哪个链路的问题，首先怀疑是客户端存在问题，针对客户端侧部署抓包程序，包含所有的客户端 Pod，客户端 Pod 所在子机/母鸡
2. 隔了一天后续现问题，但是看到客户端侧都正常，排除客户端侧问题，继续在服务侧部署抓包，抓所有 CoreDNS Pod，CoreDNS Pod 所在子机/母鸡，等待下次复现
3. 等待下次复现后，发现 CoreDNS 节点到 CoreDNS Pod 侧链路存在问题
4. 登陆 CoreDNS 节点进一步排查确认原因在于内核参数配置问题

存在大量沟通和配合工作，整体排查耗时在3d+

1. 根据用户反馈的异常 DNS 请求截图，根据 DNS 域名搜索 DNS 流日志，获取异常时间段DNS流日志，获取异常流日志
2. 根据流日志分析发现客户端Pod和客户端节点已经把包发出去，并且 CoreDNS 节点网卡也收到了包，只是 CoreDNS Pod 没有收到包，缩小问题链路为“CoreDNS Pod节点->CoreDNS Pod”，问题发生在服务端节点
3. 登陆 CoreDNS 节点进一步排查确认原因在于内核参数配置问题

Request Log											
Start Time ↑	Client	Server	Type ▾	Request Domain	Response Result	Status	Response Code	Response Desc	Enum(Tap_side)	Response Detail	
2024-05-20 11:41:11	192.168.1.101	kube-dns	AAAA	cloud-app.my	Success	Unknown	null	--	Client Process	0 js	
2024-05-20 11:41:11	192.168.1.102	kube-dns	AAAA	cloud-app.my	Success	Unknown	null	--	Client NIC	0 js	
2024-05-20 11:41:11	192.168.1.103	kube-dns	AAAA	cloud-app.my	Success	Unknown	null	--	Client K8s Node	0 js	
2024-05-20 11:41:11	192.168.1.104	kube-dns	A	cloud-app.my	Success	Unknown	null	--	Client Process	0 js	
2024-05-20 11:41:11	192.168.1.105	kube-dns	A	cloud-app.my	Success	Unknown	null	--	Client NIC	0 js	
2024-05-20 11:41:11	192.168.1.106	kube-dns	A	cloud-app.my	Success	Unknown	null	--	Client K8s Node	0 js	
2024-05-20 11:41:11	192.168.1.107	kube-dns	A	cloud-app.my	Success	Unknown	null	--	Client Process	0 js	
2024-05-20 11:41:11	192.168.1.108	kube-dns	A	cloud-app.my	Success	Unknown	null	--	Client NIC	0 js	
2024-05-20 11:41:11	192.168.1.109	kube-dns	A	cloud-app.my	Success	Unknown	null	--	Client K8s Node	0 js	
2024-05-20 11:41:11	192.168.1.110	kube-dns	A	cloud-app.my	Success	Unknown	null	--	Client Process	0 js	
2024-05-20 11:41:11	192.168.1.111	kube-dns	A	cloud-app.my	Success	Unknown	null	--	Client NIC	0 js	
2024-05-20 11:41:11	192.168.1.112	kube-dns	A	cloud-app.my	Success	Unknown	null	--	Client K8s Node	0 js	
2024-05-20 11:41:11	192.168.1.113	kube-dns	A	cloud-app.my	Success	Unknown	null	--	Server NIC	0 js	
2024-05-20 11:42:00	192.168.1.114	kube-dns	A	cloud-app.my	Success	Unknown	null	--	Client Process	0 js	
2024-05-20 11:42:00	192.168.1.115	kube-dns	A	cloud-app.my	Success	Unknown	null	--	Client NIC	0 js	
2024-05-20 11:42:00	192.168.1.116	kube-dns	A	cloud-app.my	Success	Unknown	null	--	Client K8s Node	0 js	

根据流日志直接排除3个潜在问题点，整体排查耗时在30min

05、未来展望

从被动排查到主动响应



中国计算机学会
CHINA COMPUTER FEDERATION

DeepFlow[®]



蓝鲸智云



ClickHouse

eBPF 零侵扰可观测性 Meetup

业务风险识别、主动干预

- 配置业务时延和成功率告警，联动调度提升业务可用性

TKE Serverless Pod 支持三/四层指标和流日志

- 针对业务可选开启 cBPF，优化 cBPF 内存占用



社区二维码

THANKS

感谢您的观看

Thank you for watching

赵奇圆 腾讯云



中国计算机学会
CHINA COMPUTER FEDERATION

DeepFlow[®]



蓝鲸智云



ClickHouse

eBPF 零侵扰可观测性 Meetup